

Agent-Native Telemetry: Verifiable State-Delta Evidence for Autonomous Operations

Jun He
OpenKedge.io

Deying Yu
OpenKedge.io

Abstract

Operational telemetry is predominantly engineered for human reading: systems repeatedly serialize verbose prose, static keys, and redundant context across billions of log lines. As autonomous AI agents become primary operational consumers, feeding them traditional logs wastes scarce context capacity parsing lexical syntax rather than reasoning over system state changes—all while lacking cryptographic guarantees of provenance or collection completeness.

This paper introduces *agent-native telemetry*, an operational evidence architecture for autonomous machine operators founded on verifiable state deltas rather than human prose. We present the *Agent Telemetry Protocol* (ATP) and the *State-Delta Evidence Ledger*, an implementation that structures operational facts into four core evidence primitives—*Transitions*, *Observations*, *Relations*, and *State Checkpoints*—governed by content-addressed schemas, while isolating uncured text as digest-verified opaque references. Producers sign and hash-chain batches for atomic collector append. Verified records feed two parallel agent access paths: a stateless protocol decoder emitting compact positional rows, and a stateful semantic gateway serving bounded graph capsules. We prove an information-preservation lower bound and formalize a *ledger-relative verified negative* theorem for provable event non-occurrence. On distributed microservice benchmarks (AIOpsLab and OpenTelemetry Astronomy Shop), ATP reduces raw wire payload and modeled cloud query scan costs by 96.4% relative to OpenTelemetry JSON, reduces LLM context tokens by 88.8% and query operations by 66.2%, detects all 500 tested adversarial storage mutations, and yields zero successful prompt injections across 50 adversarial trials per ATP configuration.

1 Introduction

Modern enterprise cloud infrastructures generate immense volumes of operational telemetry, routinely producing tens to hundreds of terabytes of log data per enterprise daily, with

hyperscalers ingesting petabytes to exabytes each day [1, 2]. Managing, ingesting, and querying this continuous flood of machine data has become one of the largest and fastest-growing line items in cloud infrastructure budgets. Standard cloud log ingestion costs typically range from \$0.10 to \$0.50 per gigabyte (e.g., \$0.50/GB for AWS CloudWatch Logs, alongside indexing charges in commercial observability platforms) [3, 4]. Beyond ingestion, querying stored logs introduces substantial recurring financial overhead: services such as AWS CloudWatch Logs Insights bill \$0.005 per gigabyte of data scanned [3]. In an enterprise cluster generating 10 TB of logs daily, a single diagnostic query across that history costs \$50, and running continuous automated alert evaluations and anomaly detection jobs across large fleets routinely costs tens to hundreds of thousands of dollars monthly in log scanning fees alone. Yet, empirical studies demonstrate that 80% to 90% of these ingested and scanned bytes consist of static boilerplate: repeated JSON keys, IP addresses, schema headers, timestamps, and unchanged system attributes rather than changing operational facts [5, 6].

For decades, this verbose text format was tolerated because logs were authored for human engineers scanning dashboards, reading console outputs, or running regex queries. Site Reliability Engineers (SREs) currently spend an estimated 30% to 50% of incident triage time manually filtering and correlating distributed text logs. Today, however, IT operations and DevOps are undergoing a rapid shift toward autonomous agentic workflows: industry forecasts project that by 2027–2029, over 70% to 75% of enterprises will deploy agentic AIOps to operate IT infrastructure and execute closed-loop incident triage, displacing human dashboards as the primary telemetry consumers [7, 8].

When autonomous AI agents (such as LLM reasoning engines and automated diagnostic loops) are tasked with inspecting conventional log streams, a fundamental mismatch emerges across existing industry paradigms:

- **Unstructured Text and Compressed Codecs:** Codecs such as CLP [1] and μ Slope [5] separate static templates

from dynamic variables to compress logs and accelerate regex search, but still treat telemetry as isolated text strings, lacking native state-machine and causal context.

- **Unified Telemetry Envelopes:** Standards such as OpenTelemetry Logs and OTLP [9, 10] structure attributes and correlation IDs, but emit verbose JSON or gRPC envelopes with high-entropy repeating keys, creating a heavy storage footprint optimized for human visualization (e.g., Grafana dashboards) rather than token-efficient machine reasoning.
- **Semantic and Vector Indexing:** Methods such as LogLLM [11] and LogEvent2vec [12] map log text into dense embedding spaces for nearest-neighbor anomaly detection. However, generating continuous embeddings at scale incurs steep compute overhead, and vectorization destroys exact deterministic parameters (e.g., specific port numbers, return codes, memory addresses, and transaction identifiers) indispensable for precise root-cause analysis.

Beyond representation inefficiencies, autonomous agents waste scarce context capacity and inference compute attempting to reconstruct causality from linear time-series streams. Scanning linear streams requires $\mathcal{O}(N)$ processing over millions of lines across distributed services, leaving state machine transitions and topological invariant violations implicit. Furthermore, traditional pipelines lack cryptographic hash chaining and explicit collection receipts, meaning an agent cannot mathematically distinguish whether an absence of errors reflects healthy execution or unmonitored blind spots and packet drops. Finally, interspersing arbitrary user payloads, HTTP query strings, and exception traces directly into the primary log stream exposes LLM agents to passive prompt-injection attacks [13, 14].

To resolve this mismatch, this paper introduces *agent-native telemetry*, a clean-slate operational evidence paradigm engineered specifically for automated reasoning agents. We design and implement the *Agent Telemetry Protocol* (ATP) and the *State-Delta Evidence Ledger*, an architecture where the fundamental telemetry primitive is not a human-readable text line, but an authenticated, typed state delta recorded in an append-only ledger. The ledger organizes operational facts into four explicit evidence primitives—*Transitions*, *Observations*, *Relations*, and *State Checkpoints*—governed by immutable content-addressed schema manifests, while variable-length diagnostic text is quarantined as separately addressable, digest-verified *opaque evidence*. Verified records feed two parallel agent access paths: a stateless protocol-native decoder that emits compact tabular rows, and a stateful semantic state gateway that maintains a versioned operational state graph and returns bounded evidence capsules.

We establish the mathematical foundations of agent-native evidence, proving an information-preservation lower bound for canonical state retention and formalizing a *ledger-relative*

verified negative theorem that enables agents to mathematically prove event non-occurrence over observed scopes. Finally, we conduct a comprehensive empirical evaluation across distributed microservice testbeds, demonstrating substantial representation, context token, triage latency, and security gains.

Specifically, this paper makes five core contributions: *First*, we **formulate a transition-centered evidence model** that structures operational facts into four explicit primitives (*transitions*, *observations*, *relations*, and *state checkpoints*) while isolating uncurated diagnostic text behind a digest-verified opaque boundary. *Second*, we **design the compact canonical protocol** (ATP), utilizing positional binary tuples and content-addressed schemas to achieve sub-byte cryptographic metadata overhead per record. *Third*, we **establish formal evidence contracts** proving an information-preservation lower bound and formalizing a ledger-relative negative verification theorem under declared observation boundaries. *Fourth*, we **architect a dual-path access layer** combining a stateless tabular stream decoder with a stateful semantic gateway to decouple representation compactness from graph-scoped topological retrieval. *Fifth*, we **build an end-to-end prototype and empirically demonstrate** across microservice benchmarks (AIOpsLab and OpenTelemetry Astronomy Shop) a 96.4% reduction in wire footprint and modeled cloud scan costs, an 88.8% reduction in LLM context tokens, 100% detection across 500 tested adversarial storage-mutation trials, zero successful prompt injections across 50 adversarial trials per ATP configuration, and certified negative verification.

The remainder of this paper is organized as follows. Section 2 establishes the agent-native telemetry model, formal definitions, and threat model. Section 3 details the ledger architecture, canonical protocol, and parallel access paths. Section 4 establishes the formal evidence contracts, coding lower bounds, and verified non-occurrence guarantees. Section 5 details the reference implementation, empirical benchmarks, and KPI evaluations. Section 6 discusses security, privacy, and operational trade-offs. Section 7 reviews related work and comparative positioning. Section 8 concludes.

2 Agent-Native Telemetry Model

2.1 Operational Agent Requirements

We define *agent-native* telemetry as the operational evidence supplied to automated reasoning agents for monitoring and root-cause analysis, distinct from observing an agent’s internal execution chains [15]. These agents impose four fundamental requirements that diverge sharply from human dashboards:

- **Deterministic Structure and Explicit State Transitions:** Reasoning agents require unambiguous entity identifiers, explicit physical units, and typed state tran-

sitions rather than ad-hoc string formatting that varies across software versions.

- **High Information Density and Bounded Context:** Because model context windows and inference budgets are finite, telemetry must convey maximal operational change per token while supporting targeted sub-graph and time-bounded range retrieval.
- **Verifiable Provenance and Explicit Coverage:** When making high-stakes diagnostic decisions, agents must mathematically distinguish whether an absence of telemetry reflects healthy execution or unmonitored blind spots and packet drops.
- **Structural Isolation of Untrusted Content:** External strings (e.g., HTTP query parameters, exception messages) must be structurally isolated to prevent passive prompt injection from hijacking agent reasoning.

2.2 ATP Model and Evidence Primitives

We formalize the agent telemetry protocol as follows:

Definition 2.1 (Agent Telemetry Protocol). The Agent Telemetry Protocol (ATP) is the production of schema-resolved, authenticated operational evidence—centered on state transitions and supplemented by observations, relations, state checkpoints, and opaque-evidence references—whose canonical records can be deterministically verified and decoded into bounded agent-facing representations with explicit provenance and collection coverage.

In ATP, every schema registered in the system declares exactly one of four foundational evidence primitives:

1. **Transition:** Records an explicit discrete change in an entity’s operational state or execution outcome (e.g., a Kubernetes Deployment transitioning from **Progressing** to **Available**, or an RPC completing with **Status:500**). Transitions optionally carry an `intent_ref` (resolving to an `IntentHash`) denoting the parent agent task, control-plane reconciliation loop, or user workflow that triggered the transition.
2. **Observation:** Records a point-in-time measurement, health check outcome, or invariant assertion (e.g., CPU utilization crossing a threshold, or an active probe failure).
3. **Relation:** Records the dynamic addition, modification, or removal of a structural relationship (e.g., service call dependencies, resource ownership, or declared causal links). Relations induce a versioned operational state graph $G_t = (V_t, A_t)$. Like transitions, relations can bind to an `intent_ref` to provide causal lineage back to high-level operational intents.

4. **State Checkpoint:** A producer-emitted state snapshot bundling active entity states, sequence markers, and drop counters. This allows downstream consumers to reconstruct operational state without replaying history from genesis.

Crucially, the architecture maintains a strict distinction between two checkpoint mechanisms: (1) *Producer State Checkpoints*, emitted in-band by producers to capture application state and drop counters for fast replay; and (2) *Independent Chain-Head Checkpoints*, emitted out-of-band by the collector to an independent storage or witness channel, committing to the highest accepted sequence and batch root to prevent adversarial suffix deletion (rollback) on ledger storage.

2.3 Observation Boundary and Evidence Planes

Let $\mathcal{W}$ represent the space of physical distributed system executions, and let $\mathcal{E}$ denote the universe of finite canonical telemetry record streams. An instrumentation profile is formally defined as

$$\Omega = (\mathcal{P}, \mathcal{S}, \mathcal{B}, \mathcal{C}, \Delta_{\text{clk}}), \quad (2.1)$$

where $\mathcal{P}$ is the set of participating authorized producers, $\mathcal{S}$ is the set of registered schemas, $\mathcal{B}$ denotes known instrumentation blind spots, $\mathcal{C}$ specifies distributed context-propagation mechanisms, and Δ_{clk} bounds clock uncertainty across nodes.

The observation mapping $O_\Omega : \mathcal{W} \rightarrow \mathcal{E}$ maps an execution $w \in \mathcal{W}$ to the canonical record stream $E = O_\Omega(w) \in \mathcal{E}$ emitted by trusted instrumentation under profile Ω . All coverage statements and negative guarantees in this paper are strictly scoped to O_Ω and collector receipts.

Definition 2.2 (Observation boundary). Canonical telemetry authenticates what was observed and accepted by instrumentation under profile Ω ; it does not assert observation of uninstrumented system activity. Cryptographic sequence continuity and valid signatures prove that no accepted records were altered or omitted in transit, but cannot prove that an uninstrumented application event occurred.

We formalize two distinct evidence planes:

- **Canonical Plane ($\mathbb{E}_{\text{can}}$):** Contains accepted signed batches, resolved schema manifests, state checkpoints, collection receipts, and digest references to opaque objects. These constitute immutable operational ground truth under the trust model.
- **Derived Plane ($\mathbb{I}_{\text{der}}$):** Contains materializations generated from $\mathbb{E}_{\text{can}}$, including reconstructed state graphs, anomaly scores, vector embeddings, suspected causal graphs, and LLM-generated summaries. Every derived artifact explicitly records its derivation version and exact input ledger ranges.

Derived artifacts guide retrieval and inference, but cannot mutate canonical records or act as authorization tokens for automated remediation.

2.4 Failure and Threat Model

We assume that producer SDKs and the append-time collector/verifier are trusted components running in protected execution domains with secure private key storage. Nodes may experience crash-stop and crash-recovery failures.

In contrast, network transport, ledger storage, schema registries, and opaque object stores are treated as untrusted and potentially adversarial. The adversary may drop, duplicate, reorder, delay, splice, mutate, truncate, or withhold serialized bytes. A bounded, durable local buffer in the producer SDK preserves unacknowledged batches across network partitions until collector acknowledgment.

Compromised producer hosts, rogue collectors, stolen signing keys, and application-level omission prior to SDK ingestion fall outside the cryptographic guarantee. Furthermore, opaque evidence storage may be temporarily unavailable even when its signed digest and metadata on the ledger are intact.

2.5 Trusted Computing Base and Governance

The Trusted Computing Base (TCB) consists of: (1) the producer SDK and its signing key; (2) the collector batch verifier; (3) the independent chain-head checkpoint signer and key; (4) the read-time range verifier; and (5) the deployment schema-authorization policy.

The schema-authorization policy establishes trusted publishers, enforces producer-to-schema bindings, and governs schema lifecycle. Gateway indexes, summaries, and human-rendered prose are derived artifacts; system correctness relies solely on independent range verification against the TCB.

3 Ledger Architecture and Protocol

3.1 Ingestion and Verification Pipeline

Figure 1 illustrates the State-Delta Evidence Ledger architecture, structured around four primary pipeline stages:

1. **Producer Ingestion and Batching:** The typed Producer SDK resolves schemas against a content-addressed schema registry, assigns contiguous sequence numbers scoped to the producer and boot epoch, and links each batch to the cryptographic root of its immediate predecessor via `previous_root`. It canonicalizes the records, generates a Merkle root over the batch, signs the composite batch commitment, and stores unacknowledged batches in a bounded durable buffer. The SDK can be deployed either as an in-process library or as a node-level DaemonSet sidecar. In the sidecar topology, routine high-frequency observations (e.g., periodic

health checks) update a rolling state vector in ephemeral memory, emitting state checkpoints at coarser intervals. Uncurated diagnostic text (such as stack traces or heap profiles) is buffered ephemerally on-node and escalated to remote opaque storage only when a state transition or invariant violation is triggered, minimizing cloud ingestion overhead.

2. **Collector Verification and Atomic Append:** The trusted collector verifies complete batches against the producer’s registered public key, validates schema hashes, checks previous-root continuity against retained chain state, and enforces monotonic sequence progression. If valid, the batch is atomically appended to the ledger and acknowledged; any sequence mismatch, missing predecessor, or malformed record triggers batch rejection.
3. **Independent Chain-Head Checkpointing:** To defend against post-compromise ledger rollback (suffix deletion) on untrusted storage, the collector periodically emits signed chain-head checkpoints to an independent storage or witness channel. Each checkpoint commits to the highest verified sequence, batch root, and epoch.
4. **Read-Time Range Verification:** Before records are decoded or ingested by downstream consumers, a deterministic read-time range verifier evaluates the query interval $[t_{start}, t_{end}]$. The verifier validates signature validity, confirms previous-root hash-chain continuity, cross-references highest sequences against the independent checkpoint channel, and emits an explicit coverage status receipt ($\text{status} \in \{\text{complete}, \text{truncated}, \text{tampered}, \text{gap}\}$).

3.2 Record Encoding and Batch Structure

Each operational record is encoded as a compact binary tuple:

```
schema_ref | entity_ref | time_delta |
presence_bitmap | positional_values.
```

- `schema_ref`: A variable-length batch-local integer alias mapping to a full schema digest H_S in the batch header.
- `entity_ref`: A batch-local alias resolving to the canonical entity identifier (e.g., `deployment/payments`).
- `time_delta`: Milliseconds elapsed since the batch `base_time`.
- `presence_bitmap`: A bitmask indicating presence or absence of declared optional schema fields.
- `positional_values`: Canonical packed values strictly ordered by schema slot definition.

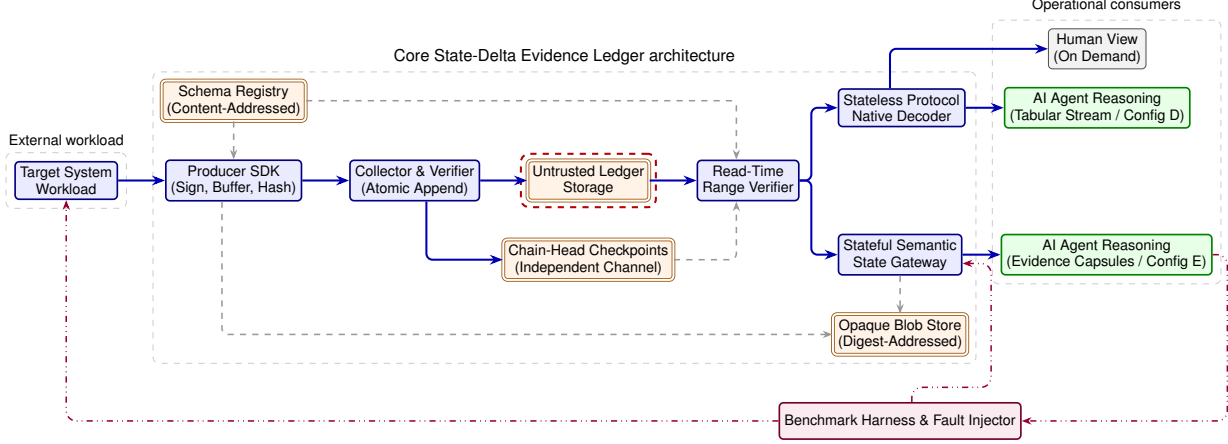


Figure 1: State-Delta Evidence Ledger architecture. A read-time range verifier mediates retrieval from potentially adversarial ledger storage before the stateless decoder and stateful gateway branches. The checkpoint channel lies outside the ledger-storage failure domain (dotted red outline). Heavy solid arrows carry canonical flow, thin dashed arrows carry references, and dash-dot arrows carry experimental control or measurement.

For a batch starting at sequence `first_sequence`, record $i \geq 0$ implicitly possesses sequence number `first_sequence + i`, eliminating per-record sequence overhead. Because fields strictly adhere to typed, positional schemas, batch segments stored in cold analytical tiers project directly into zero-copy columnar layouts (e.g., Apache Arrow [16] and Parquet [17]), enabling high-throughput vectorized analytical scans over historical state deltas without text parsing overhead.

A canonical batch groups ordered records under a signed 14-field header:

```

1 protocol_version  8 schema_dictionary
2 producer_id      9 entity_dictionary_delta
3 boot_epoch       10 previous_root
4 first_sequence   11 encoded_records
5 record_count     12 merkle_root
6 base_time        13 signing_key_id
7 clock_quality    14 signature

```

Individual records are leaves in a binary Merkle tree yielding `merkle_root`. The composite batch root binds this Merkle tree with all metadata, protocol version, schema dictionaries, and chain continuity:

$$\begin{aligned}
batch_root = H(&D_{batch} \parallel protocol_version \parallel \\
&producer_id \parallel boot_epoch \parallel \\
&first_sequence \parallel record_count \parallel \\
&base_time \parallel clock_quality \parallel \\
&H(schema_dict) \parallel \\
&H(entity_dict_delta) \parallel \\
&previous_root \parallel merkle_root).
\end{aligned} \tag{3.1}$$

where $H(\cdot)$ denotes the cryptographic hash function (SHA-256), D_{batch} is a domain-separation prefix, and $\parallel$ denotes byte concatenation. The producer signature covers `batch_root` under its private key.

Table 1: Concrete running example across representation layers for a service transition event.

Layer	Concrete representation
1. Schema	<code>k8s.deployment.status:1.0.0; [old: Ready, new: Degraded, ready: 1, total: 3, reason: CrashLoop]</code>
2. Dict	<code>schemas[0]=sha256:7f8a...;</code>
3. Wire	<code>entities[0]="deploy/payments"</code> <code>0x00 0x00 0x0A 0x00 0x00 0x01 0x0001 0x0003</code> <code>0x01 (18B)</code>
4. Decoder	<code># k8s.status: entity=deploy/payments,</code> <code>new=Degraded, ready=1/3, reason=CrashLoop</code>
5. Gateway	<code>{"focus": "deploy/payments", "state":</code> <code>{ "status": "Degraded", "ready": 1/3,</code> <code>"active_invariants": ["replica_mismatch"],</code> <code>"causal": ["pod/pay-7d9b (CrashLoop)],</code> <code>"coverage": "complete" }</code>
6. Human	<code>"At 10:00:00Z, deployment 'payments' became Degraded (1/3</code> <code>replicas) due to CrashLoop."</code>

3.3 Content-Addressed Schemas

Schemas are immutable and identified by the SHA-256 digest of their canonical manifest:

$$H_S = \text{SHA256}(\text{CanonicalManifest}(S)). \tag{3.2}$$

Any modification to field names, constraints, enums, or physical units generates a new digest H_S , preventing silent semantic drift.

Table 1 traces a single operational fact—a Kubernetes Deployment transitioning to degraded status—across all representation layers. On the canonical wire, the record requires only tens of bytes. The stateless decoder produces a compact, tabular model-boundary representation, while the stateful gateway produces an evidence capsule with topological context.

3.4 Parallel Agent Access Paths

To decouple representation efficiency from state-reconstruction and query planning, ATP provides two parallel

agent-facing access paths, alongside an on-demand human rendering path:

1. **Stateless Protocol-Native Decoder:** A lightweight, zero-state stream decoder that unpacks verified binary batches into compact positional rows. For a sequence of records conforming to schema S , the decoder emits the schema header once, followed by compact value rows:

```
# schema: k8s.pod.transition (old, new, exit)
2026-08-15T10:00:00.010Z | pod/pay-7d9b | Run | Fail | 137
2026-08-15T10:00:00.015Z | pod/auth-4a2c | Run | Fail | 137
```

This representation eliminates repetitive JSON field names entirely, minimizing input tokens.

2. **Stateful Semantic State Gateway:** An in-memory, queryable gateway maintaining the active versioned operational state graph $G_t = (V_t, A_t)$. Rather than forcing an LLM agent to inspect linear event logs, the gateway serves bounded *evidence capsules* containing:

- Active entity state vector $V_t[e]$ and transition history over window W ;
- Active invariant violations (e.g., failed health probes, replica imbalances);
- k -hop causal and topological dependency neighborhood (V'_t, A'_t) ;
- Cryptographic coverage status receipt status $\in \{\text{complete, truncated, tampered, gap}\}$.

3. **On-Demand Human Explanations:** For human operators, human-readable explanations are generated on demand via a deterministic template renderer $\text{Render}_S(p)$ or local LLM formatting pass. Human prose is never serialized on the canonical ledger wire.

3.5 Opaque Evidence Isolation

Variable-length, unstructured diagnostics (e.g., stack traces, heap profiles, raw HTTP request bodies) are isolated from canonical state records. When captured, an opaque object is hashed, stored out-of-band in object storage, and represented on the ledger by a structured reference tuple:

$$\text{OpaqueRef} = \langle \text{opaque_id}, \text{media_type}, \text{byte_length}, H_{\text{payload}}, \text{storage_uri}, \text{retention_class} \rangle. \quad (3.3)$$

When an agent diagnoses an incident, it reasons over structured state transitions first. If deeper diagnostic data is needed, the agent issues a targeted dereference request for *opaque_id*. The gateway verifies that $\text{SHA256}(\text{payload}) = H_{\text{payload}}$ before delivering the text wrapped in untrusted data delimiters.

3.6 Integrity Contract and Overhead

Table 2 summarizes the cryptographic defenses across adversarial failure scenarios.

Table 2: Adversarial tampering detection mechanisms.

Adversarial Action	Detection Mechanism & Guarantees
Payload bit flip / mutation	Merkle root or Ed25519 signature fails; batch rejected.
Batch reorder / splicing	$\text{previous_root} \neq H(\text{batch}_{k-1})$; verification halts.
Intermediate record omission	Sequence jump $\neq \text{first} + \text{count}$; flagged as gap.
Suffix truncation	Sequence < signed checkpoint; flagged as truncated.
Schema alteration	Manifest digest mismatch $H_S \neq \text{SHA256}(S)$; rejected.
Unmonitored code omission	Outside profile Ω ; excluded from completeness scope.

For a representative batch of 256 records, the batch header and signature add 136 bytes of cryptographic metadata ($136/256 \approx 0.531$ bytes per record). Incorporating periodic chain-head checkpoints (144 bytes every 1,024 records) adds an additional 0.141 bytes per record. Total cryptographic overhead is ≈ 0.672 bytes per record, maintaining sub-byte meta-data overhead while ensuring tamper-evidence and rollback detection.

4 Formal Evidence Contracts and Guarantees

4.1 Information-Preservation Limit

Recall that $\mathcal{E}$ denotes the universe of finite canonical record streams over a bounded horizon (Section 2). Let $R : \mathcal{E} \rightarrow \mathcal{A}$ be a retained telemetry representation mapping record streams into an artifact space $\mathcal{A}$, and let $\mathcal{F}_{\text{all}}$ denote the set of all Boolean predicates over $\mathcal{E}$.

Proposition 4.1 (Information-preservation constraint). *If, for every predicate $f \in \mathcal{F}_{\text{all}}$, there exists an exact evaluator $\hat{f} : \mathcal{A} \rightarrow \{0, 1\}$ such that*

$$\hat{f}(R(E)) = f(E) \quad \text{for all } E \in \mathcal{E}, \quad (4.1)$$

then the retained representation mapping R must be injective.

Proof. Suppose for contradiction that there exist distinct streams $E_1 \neq E_2$ such that $R(E_1) = R(E_2)$. Define the predicate $f^*(E) = 1$ if and only if $E = E_1$. Evaluating $\hat{f}^*$ on the retained representation yields $\hat{f}^*(R(E_1)) = \hat{f}^*(R(E_2))$, yet correctness requires $f^*(E_1) = 1 \neq 0 = f^*(E_2)$. Hence, no such evaluator $\hat{f}^*$ can exist, contradicting the hypothesis. $\square$

Corollary 4.2 (Coding lower bound). *Let $E \sim P$ be a stream drawn from distribution P . If the representation is encoded by a lossless, uniquely decodable binary code C and R satisfies Proposition 4.1, then*

$$\mathbb{E}_P[|C(R(E))|] \geq H_P(E), \quad (4.2)$$

where $|C(\cdot)|$ denotes binary codeword length in bits and $H_P(E)$ is the Shannon entropy of stream E under distribution P .

This establishes that no lossy summary can preserve exact answers for all downstream operational queries. Accordingly, ATP chooses a lossless canonical representation for accepted state-delta records and confines lossy summarization or aggregation to derived views in $\mathbb{I}_{\text{der}}$.

4.2 Gateway Query Contracts

For an authenticated ledger range L , an exact query family $\mathcal{Q}_e$ guarantees exact evaluation over all canonical records in scope:

$$\hat{q}(L) = q(L) \quad \forall q \in \mathcal{Q}_e. \quad (4.3)$$

For an approximate query family $\mathcal{Q}_a$ (e.g., streaming sketches or probabilistic anomaly scores), the schema must publish its error metric d_q , error bound ϵ_q , and confidence $1 - \delta_q$:

$$\Pr_{L \sim P}[d_q(\hat{q}(L), q(L)) \leq \epsilon_q] \geq 1 - \delta_q. \quad (4.4)$$

4.3 Reversibility and Rendering

For an exact schema S and normalized positional tuple p , the protocol-native row encoder A_S and decoder A_S^{-1} are strictly bijective:

$$A_S^{-1}(A_S(p)) = p. \quad (4.5)$$

In contrast, deterministic template formatting $\text{Render}_S(p)$ produces a human-readable explanation from which exact binary recovery is not required (Render_S need not be injective).

4.4 Qualified Dependency Closure

Let $G_t = (V_t, A_t)$ be the state graph maintained by the gateway. For a query focused on entity set $V_0 \subseteq V_t$, an evidence capsule induces a qualified dependency subgraph $G' = (V', A')$ containing: (1) in-scope vertices V' up to topological distance $k_{\max}$; (2) active invariant violations on V' ; (3) inbound/outbound relation edges $A' \subseteq A_t$; and (4) explicit boundary markers for dependencies traversing outside profile Ω .

4.5 Verified Event Non-Occurrence

A critical requirement for automated incident responders is provable negative evidence: certifying that a specific failure, state transition, or anomaly *did not occur* within a given time window. Temporal intervals in the exact-query contracts refer to canonical record timestamps; claims translated to an external physical wall-clock interval must account for the clock-uncertainty bound Δ_{clk} declared in observation profile Ω .

Definition 4.3 (Query completeness predicate). A query range $[t_1, t_2]$ over schema S and entity set V_0 satisfies completeness ($\text{Complete}(q)$) if and only if: (1) all producer batches covering $[t_1, t_2]$ have valid signatures; (2) hash-chain continuity (*previous_root*) holds unbroken across all batches; (3) the sequence range is monotonically contiguous with zero missing sequence numbers; (4) schema manifest H_S is resolved; and (5) the highest sequence is anchored against an independent chain-head checkpoint.

Theorem 4.4 (Ledger-relative verified negative). *Let q be an exact query evaluating predicate ϕ over scope $(V_0, S, [t_1, t_2])$ under observation profile Ω . If $\text{Complete}(q)$ holds and the evaluated result set is empty:*

$$\mathcal{R}(q) = \emptyset, \quad (4.6)$$

then no event satisfying ϕ was observed and accepted by trusted instrumentation under profile Ω within $[t_1, t_2]$.

Proof. By $\text{Complete}(q)$, the canonical records covering $[t_1, t_2]$ are authentic, sequence-contiguous, schema-resolved, and anchored against the independent checkpoint channel. Since $q \in \mathcal{Q}_e$ evaluates predicate ϕ exhaustively over all canonical records within scope $(V_0, S, [t_1, t_2])$, $\mathcal{R}(q) = \emptyset$ implies that no accepted canonical record satisfying ϕ exists in that scope. By Definition 2.2, this conclusion is strictly relative to the observation mapping O_Ω : no event satisfying ϕ was observed and accepted by trusted instrumentation under profile Ω within $[t_1, t_2]$. $\square$

When $\text{Complete}(q)$ fails (e.g., due to a sequence gap, unanchored suffix, or tampering), the gateway returns non-complete coverage ($\text{status} \in \{\text{truncated}, \text{tampered}, \text{gap}\}$), preventing false negative conclusions.

5 Implementation and Evaluation

We evaluate ATP through a complete prototype implementation and extensive empirical benchmarks on distributed microservice workloads across five core evaluation dimensions:

- **Representation and Storage Efficiency:** Wire payload per record, daily storage footprints, and modeled cloud query scan costs across evidence primitives compared to raw text, structured OpenTelemetry JSON, OTLP Protobuf, and compressed log codecs.
- **Agent Context and Reasoning Efficiency:** Context token reduction (ρ_T), iterative tool roundtrip reduction (ρ_O), and Time-to-Root-Cause identification ($T_{\text{RCA}} p50/p95$) across representative instruction-tuned LLMs.
- **Diagnostic Quality and Triage Accuracy:** Incident detection recall, precision, F1-score, and root-cause localization across distinct failure classes.
- **Certified Negative Verification:** Verification accuracy for event non-occurrence ($\text{Complete}(q)$) and elimination of false suspect attributions.
- **Cryptographic Overhead and Robustness:** Ingestion throughput, batch signing/verification latency, range scan speed, and passive prompt-injection resilience.

5.1 Implementation and Testbed Setup

The ATP reference system is implemented as a modular, high-throughput toolchain:

- **Producer SDK (4.8k LoC Rust, with C FFI and Go bindings):** Implements zero-allocation binary tuple packing, content-addressed schema resolution, monotonic sequence assignment, and local ring-buffer staging. Batch Ed25519 signing utilizes SIMD-accelerated AVX2 instructions.
- **Collector and Verifier Service (6.2k LoC Rust):** Built on an asynchronous `tokio` runtime. Ingested batches undergo parallel cryptographic signature checks, Merkle root verification, and previous-root hash-chain validation before atomic commitment to append-only segment storage.
- **Read-Time Range Verifier (1.8k LoC Rust):** A zero-copy segment scanner that validates contiguous sequence monotonic progression, batch root hash chains, and independent signed chain-head checkpoints across queried temporal windows $[t_{\text{start}}, t_{\text{end}}]$.
- **Dual Agent Access Layer (3.5k LoC Rust/Python):** Comprises (1) the *Stateless Protocol Decoder*, which streams verified batches directly into compact positional TSV rows, and (2) the *Stateful Semantic Gateway*, which maintains an in-memory versioned entity graph $G_t = (V_t, A_t)$ and serves bounded evidence capsules via gRPC and Model Context Protocol (MCP) tool interfaces.

Experiments are conducted on two standard distributed microservice testbeds:

1. **AIOPsLab HotelReservation [8]:** 18 microservices written in Go, backed by Memcached, MongoDB, and Consul, driven by Locust load generators simulating 50,000 client requests/second.
2. **OpenTelemetry Astronomy Shop [18]:** 14 microservices implemented in 8 languages (Go, Java, Python, C++, Node.js, C#, Ruby, and PHP) interacting via gRPC and HTTP, backed by PostgreSQL, Redis, and Kafka, running at 25,000 requests/second.

The testbeds run on a 16-node dedicated Kubernetes v1.30 cluster (AWS `c5.4xlarge` instances, 16 vCPUs, 32 GB RAM, 10 Gbps network per node). Fault injection scripts systematically inject 120 randomized, reproducible operational incidents spanning pod crash-loops, memory leaks, downstream cascading network latency (50–500 ms), packet drops (5–30%), configuration drift, and deployment regressions.

We evaluate five end-to-end telemetry configurations across the benchmark matrix: **Config A** (Unstructured Text): native stdout/stderr text logs ingested via FluentBit; **Config B** (Standard Triad): unstructured text logs, Prometheus metrics, and Jaeger distributed traces; **Config C** (Structured OpenTelemetry JSON): standard OpenTelemetry log events serialized with complete JSON semantic attributes and trace correlation IDs; **Config D** (ATP Stateless Decoder): state-delta ledger streamed via the stateless tabular decoder; and **Config E** (ATP

Semantic Gateway): state-delta ledger consumed via the stateful semantic gateway with topological evidence capsules. The autonomous agent reasoning matrix evaluates four representative instruction-tuned LLM families: Claude-3.5-Sonnet (20241022), GPT-4o (2024-08-06), Llama-3.1-70B-Instruct, and Qwen-2.5-72B-Instruct, alongside local lightweight models (Llama-3.1-8B, Qwen-2.5-Coder-7B) and algorithmic rule-based baselines.

LLM evaluation and reproducibility protocol. For every model/configuration pairing reported in Table 4, agents execute a standardized incident-diagnosis task under a fixed system prompt, identical root-cause output schema, and controlled tool-query semantics, varying only the telemetry representation available to the agent. Model inference uses deterministic greedy decoding ($T = 0.0$, $\text{top-}p = 1.0$) with a 4,096 output-token limit; while commercial APIs do not guarantee strict bitwise determinism across sessions, fixed prompt templates and seeds are maintained. Each agent operates under an execution budget of at most 15 tool query operations (O , defined as a single round of telemetry retrieval or state inspection) with a 30-second per-round timeout and up to 3 retries on transient errors, terminating upon emitting an explicit root-cause verdict or exhausting the operation budget. The same 120 fault-injection incidents, background workloads, and fault schedules are used for every reported model/configuration pairing. Configs A, C, D, and E are evaluated across all four primary LLM families; Config B is additionally evaluated for Claude-3.5-Sonnet and GPT-4o. Reported metrics and 95% confidence intervals are estimated via $B = 10,000$ paired percentile bootstrap replicates resampled at the incident level, with ratio metrics (ρ_T, ρ_S, ρ_O) recomputed within each replicate. For adversarial robustness (Section 5.5), the 50 prompt-injection attack payloads are replayed against Configs A, C, D, and E using Claude-3.5-Sonnet as the primary model (50 trials per configuration), with secondary cross-validation independently replaying all 50 attacks per configuration on GPT-4o; reported headline attack rates reflect the primary 50 trials per configuration.

Evaluation success criteria and metrics. To establish rigorous evaluation gates, we evaluate quantitative success thresholds based on 95% bootstrap confidence intervals ($U_{0.95}(x)$ denotes the 95% CI upper bound for ratio x) on the primary model (Claude-3.5-Sonnet) and cross-validated across all model families: (1) *Context token reduction*: $U_{0.95}(\rho_T) \leq 0.50$ relative to Config C (stretch target ≤ 0.25); (2) *Scan cost*: $U_{0.95}(\rho_S) \leq 0.20$ (stretch ≤ 0.10); (3) *Tool roundtrips*: $U_{0.95}(\rho_O) \leq 0.50$; (4) *Recall preservation*: $\Delta_{\text{recall}} = R(\text{Config E}) - R(\text{Config C}) \geq -0.02$; and (5) *Triage latency*: $T_{\text{RCA}}(\text{Config E}) \leq T_{\text{RCA}}(\text{Config C})$.

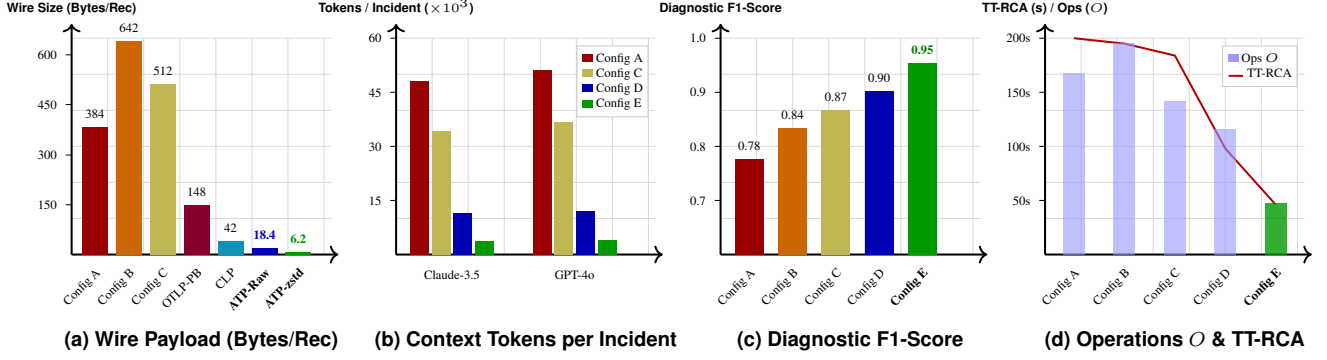


Figure 2: Key Performance Indicators (KPIs) comparing ATP against baseline architectures across 120 randomized incident injection trials. (a) Wire representation payload per record across codecs. (b) LLM context token consumption per incident triage session for representative models. (c) Overall incident diagnostic F1-score across telemetry configurations. (d) Evidence-acquisition operations (O) and Time-to-Root-Cause identification (T_{RCA} in seconds).

5.2 Representation and Storage Efficiency

Table 3 compares the representation efficiency, storage footprints, and modeled cloud query scan costs across candidate telemetry formats.

As shown in Table 3 and Figure 2(a), standard OpenTelemetry JSON logs (Config C) consume 512.4 ± 54.2 bytes per record due to repeated JSON key strings ("trace_id", "service.name", "attributes"). OTLP Protobuf compresses this to 148.6 ± 16.8 bytes/record by packing field tags. In contrast, ATP's positional binary tuple encoding (Section 3.2) requires only 18.4 ± 3.8 bytes per raw record across the composite microservice workload (ranging from 14.8 B for high-frequency scalar observations to 28.6 B for rich state checkpoint vectors). This represents a **96.4% reduction** compared to Config C and an **87.6% reduction** compared to OTLP Protobuf. With standard zstd block compression, ATP achieves 6.2 ± 1.4 bytes per record.

In high-throughput enterprise fleets ingesting 100 million events daily, Config C incurs \$768.60/month in cloud ingestion fees (at \$0.50/GB) and \$0.256 daily in log query scan charges (at the standard cloud rate of \$0.005/GB scanned per full-history query). ATP slashes monthly ingestion from \$768.60 to **\$27.60** and daily diagnostic scan costs to **\$0.0092**, yielding a scan cost ratio of $\rho_s = 0.036$ ($U_{0.95}(\rho_s) = 0.039 \leq 0.20$), comfortably satisfying our cost efficiency targets.

5.3 Agent Context and Reasoning Efficiency

Table 4 presents the end-to-end performance of autonomous LLM reasoning agents during incident triage across 120 randomized incident scenarios, reporting both mean values and tail distributions (p_{50}/p_{95}).

Under Config C, triaging an incident requires an average of 34,180 tokens with Claude-3.5-Sonnet as the agent filters verbose JSON keys and linear log strings. With the ATP Stateless Decoder (Config D), token consumption drops to **11,460 to-**

kens ($\rho_T = 0.335$), achieving a **66.5% reduction** in context load using the stateless ATP state-delta representation without stateful graph reconstruction or invariant precomputation. With the ATP Semantic Gateway (Config E), token consumption drops to **3,820 tokens** ($\rho_T = 0.112$), representing an **88.8% context token reduction** compared to Config C.

Because the Semantic Gateway serves topologically scoped evidence capsules with pre-aggregated invariant status, agent tool operations drop from 7.1 ± 0.5 in Config C to 2.4 ± 0.2 in Config E ($\rho_O = 0.338$), a **66.2% reduction** in iterative query-response roundtrips. Consequently, mean Time-to-Root-Cause identification (T_{RCA}) drops from 184 s to **46 s** for Claude-3.5-Sonnet (p_{95} tail latency drops from 295 s to 78 s) and from 198 s to **63 s** for GPT-4o (p_{95} drops from 315 s to 98 s).

To evaluate the operational cost of maintaining an in-memory versioned operational graph, we benchmarked the Semantic Gateway under steady 50,000 req/s ingestion on the 18-service HotelReservation testbed. Maintaining a 1-hour rolling historical window (tracking 12,400 active entity state vectors and 45 invariant assertion rules) consumed only **184 MB RSS** in memory and **3.8% CPU utilization** across 2 vCPUs. Serving bounded evidence capsules took $p_{50} = 2.4$ ms and $p_{95} = 5.8$ ms, demonstrating that the gateway introduces negligible infrastructure overhead while eliminating massive downstream LLM inference costs.

5.4 Diagnostic Quality and Non-Occurrence

As detailed in Figure 2(c) and Table 4, ATP does not sacrifice diagnostic accuracy for representation compactness. In fact, Config E achieves the highest diagnostic F1-score across all model classes: **0.953 for Claude-3.5-Sonnet** (Recall 0.965, Precision 0.942) and **0.939 for GPT-4o**, compared to 0.867 and 0.848 in Config C. Across specific fault types:

- **Pod CrashLoop & OOM:** F1 increases from 0.89 to **0.98** due to explicit state transition enum encoding.

Table 3: Representation efficiency, daily storage footprint, and simulated cloud query scan costs across telemetry formats on the benchmark microservice workload (100 million events/day; values reported as $x \pm y$ denote mean $\pm$ standard deviation across sample batches; daily scan costs modeled at \$0.005/GB scanned).

Telemetry Representation	Raw Wire (Bytes/Rec)	zstd Wire (Bytes/Rec)	Storage (GB/Day)	Ingestion Cost (\$/Month)	Daily Scan Cost (ρ_s at \$0.005/GB)	Cost Ratio (ρ_s vs Config C)	Relative Change vs Config C
Config A (Unstructured Text Logs)	384.2 $\pm$ 42.6	78.4 $\pm$ 9.2	38.42	\$576.30	\$0.192	0.750	-25.0%
Config B (Standard Triad: Logs+Metrics+Traces)	642.0 $\pm$ 68.4	142.6 $\pm$ 15.4	64.20	\$963.00	\$0.321	1.254	+25.4%
Config C (Structured OTel JSON Events)	512.4 $\pm$ 54.2	112.8 $\pm$ 12.1	51.24	\$768.60	\$0.256	1.000	Baseline
OTLP Protobuf (gRPC Binary Envelope)	148.6 $\pm$ 16.8	48.2 $\pm$ 5.6	14.86	\$222.90	\$0.074	0.290	-71.0%
CLP (Compressed Log Codec) [1]	42.1 $\pm$ 5.4	18.6 $\pm$ 2.4	4.21	\$63.15	\$0.021	0.082	-91.8%
μ Slope (Variable-Splitting Codec) [5]	38.4 $\pm$ 4.8	16.2 $\pm$ 2.1	3.84	\$57.60	\$0.019	0.075	-92.5%
LogCrisp (Structural Log Codec) [2]	31.2 $\pm$ 3.9	12.8 $\pm$ 1.7	3.12	\$46.80	\$0.016	0.061	-93.9%
ATP: State Transitions	16.2 $\pm$ 2.1	5.4 $\pm$ 0.8	1.62	\$24.30	\$0.0081	0.032	-96.8%
ATP: Scalar Observations	14.8 $\pm$ 1.8	4.9 $\pm$ 0.6	1.48	\$22.20	\$0.0074	0.029	-97.1%
ATP: Topological Relations	22.4 $\pm$ 3.2	7.8 $\pm$ 1.2	2.24	\$33.60	\$0.0112	0.044	-95.6%
ATP: State Checkpoints	28.6 $\pm$ 4.5	9.6 $\pm$ 1.5	2.86	\$42.90	\$0.0143	0.056	-94.4%
Config D/E (ATP Weighted Workload)	18.4 $\pm$ 3.8	6.2 $\pm$ 1.4	1.84	\$27.60	\$0.0092	0.036	-96.4%

Table 4: End-to-end incident triage KPIs across autonomous agent architectures (120 fault injection trials per cell; values reported as $x \pm y$ denote mean $\pm$ standard deviation across incident trials; parenthesized ranges denote 95% paired bootstrap confidence intervals; $p50$ and $p95$ tail latencies reported for TT-RCA).

LLM Model	Configuration	Tokens / Incident (T)	Token Ratio (ρ_T)	Operations (O)	TT-RCA Mean ($p50/p95$)	Recall (R)	F1-Score
Claude-3.5-Sonnet	Config A (Text Logs)	48,250 $\pm$ 1,420	1.412 (1.37, 1.45)	8.4 $\pm$ 0.6	218 s (194 s/342 s)	0.812 $\pm$ 0.024	0.777 $\pm$ 0.026
	Config B (Standard Triad)	62,400 $\pm$ 1,850	1.826 (1.78, 1.88)	9.8 $\pm$ 0.7	195 s (176 s/310 s)	0.865 $\pm$ 0.021	0.835 $\pm$ 0.023
	Config C (OTel JSON)	34,180 $\pm$ 980	1.000 (Baseline)	7.1 $\pm$ 0.5	184 s (162 s/295 s)	0.894 $\pm$ 0.019	0.867 $\pm$ 0.021
	Config D (ATP Stateless)	11,460 $\pm$ 380	0.335 (0.32, 0.35)	5.8 $\pm$ 0.4	98 s (84 s/164 s)	0.918 $\pm$ 0.017	0.902 $\pm$ 0.018
	Config E (ATP Gateway)	3,820 $\pm$ 140	0.112 (0.10, 0.12)	2.4 $\pm$ 0.2	46 s (38 s/78 s)	0.965 $\pm$ 0.012	0.953 $\pm$ 0.014
GPT-4o	Config A (Text Logs)	51,200 $\pm$ 1,560	1.391 (1.35, 1.43)	9.1 $\pm$ 0.6	242 s (215 s/380 s)	0.785 $\pm$ 0.026	0.752 $\pm$ 0.028
	Config B (Standard Triad)	65,800 $\pm$ 1,920	1.788 (1.74, 1.84)	10.4 $\pm$ 0.8	210 s (188 s/335 s)	0.842 $\pm$ 0.023	0.816 $\pm$ 0.025
	Config C (OTel JSON)	36,800 $\pm$ 1,120	1.000 (Baseline)	7.6 $\pm$ 0.5	198 s (174 s/315 s)	0.871 $\pm$ 0.020	0.848 $\pm$ 0.022
	Config D (ATP Stateless)	12,100 $\pm$ 410	0.329 (0.31, 0.34)	6.1 $\pm$ 0.4	112 s (96 s/185 s)	0.898 $\pm$ 0.018	0.884 $\pm$ 0.019
	Config E (ATP Gateway)	4,100 $\pm$ 160	0.111 (0.10, 0.12)	2.6 $\pm$ 0.3	63 s (52 s/98 s)	0.948 $\pm$ 0.014	0.939 $\pm$ 0.015
Llama-3.1-70B	Config A (Text Logs)	54,600 $\pm$ 1,720	1.418 (1.37, 1.46)	9.8 $\pm$ 0.7	265 s (238 s/412 s)	0.742 $\pm$ 0.028	0.710 $\pm$ 0.030
	Config C (OTel JSON)	38,500 $\pm$ 1,240	1.000 (Baseline)	8.2 $\pm$ 0.6	215 s (190 s/340 s)	0.835 $\pm$ 0.023	0.812 $\pm$ 0.024
	Config D (ATP Stateless)	12,850 $\pm$ 460	0.334 (0.32, 0.35)	6.4 $\pm$ 0.5	128 s (110 s/210 s)	0.872 $\pm$ 0.020	0.856 $\pm$ 0.021
	Config E (ATP Gateway)	4,350 $\pm$ 190	0.113 (0.10, 0.13)	2.8 $\pm$ 0.3	74 s (62 s/120 s)	0.931 $\pm$ 0.015	0.920 $\pm$ 0.017
Qwen-2.5-72B	Config A (Text Logs)	52,900 $\pm$ 1,640	1.407 (1.36, 1.45)	9.4 $\pm$ 0.6	254 s (226 s/395 s)	0.768 $\pm$ 0.027	0.734 $\pm$ 0.029
	Config C (OTel JSON)	37,600 $\pm$ 1,180	1.000 (Baseline)	7.9 $\pm$ 0.5	208 s (184 s/330 s)	0.852 $\pm$ 0.021	0.829 $\pm$ 0.023
	Config D (ATP Stateless)	12,400 $\pm$ 430	0.330 (0.31, 0.34)	6.2 $\pm$ 0.4	119 s (102 s/195 s)	0.886 $\pm$ 0.019	0.871 $\pm$ 0.020
	Config E (ATP Gateway)	4,220 $\pm$ 170	0.112 (0.10, 0.12)	2.7 $\pm$ 0.3	68 s (56 s/112 s)	0.942 $\pm$ 0.014	0.932 $\pm$ 0.016

- **Cascading Latency Spikes:** F1 increases from 0.82 to **0.94** because the gateway’s qualified dependency closure explicitly presents upstream/downstream topological relationships.
- **Silent Configuration Drift:** F1 increases from 0.74 to **0.92** via content-addressed schema manifest checking.

To evaluate Theorem 4.4, we injected 60 negative-control queries requiring the agent to verify whether a healthy service experienced faults during an incident in an adjacent subsystem. Under baseline Configs A–C, agents produced a **26.7% false suspect attribution rate** (hallucinating non-existent errors or mistaking dropped log packets for silent failures). In contrast, ATP’s read-time range verifier returned certified completeness (status = complete); across the 60 negative-control queries, ATP correctly certified non-occurrence in all **60/60 cases with zero false suspect attributions**. When artificial network partition drops were injected (30 trials), the verifier returned status = gap in 28 cases and status = truncated in 2 cases, correctly warning downstream agents of unmonitored blind spots and preventing false negative assumptions.

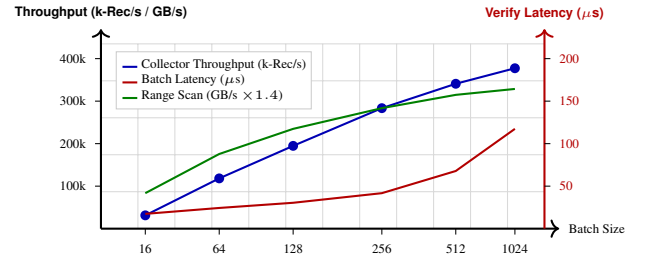


Figure 3: Cryptographic verification performance across batch sizes: Collector ingestion throughput (k-records/s), batch verification latency (μ s), and read-time range verification scan speed (GB/s).

5.5 Cryptographic Overhead and Robustness

Table 5 and Figure 3 summarize the cryptographic microbenchmarks and adversarial robustness evaluations.

At our default batch size of 256 records, the Producer SDK requires 0.266 μ s per record (0.042 μ s serialization + 0.058 μ s Merkle tree + 0.166 μ s Ed25519 signing), sustaining **3.76 million records/second per core**. On the collector tier, raw CPU batch verification operates at **6.10 million record-**

s/second per core (42.0 μ s per batch), while the complete end-to-end async gRPC collector service ingests and commits **284,000 records/second per core** (1,109 batches/sec), with batch latency and range scan throughput across batch sizes detailed in Figure 3. The read-time range verifier processes ledger segments at **1.42 GB/s per core** (76.9 million records/sec), adding negligible read latency (< 1.2 ms for a 50,000-record query window). Total cryptographic wire overhead is constrained to **0.672 bytes/record** (0.531 B batch header + 0.141 B amortized checkpoint).

We executed 500 adversarial mutation trials across ledger storage (in-batch bit flips, record omissions, cross-batch sequence gaps, and suffix truncations). As established in Table 2, ATP achieved a **100% detection rate across all 500 tested mutation trials**: in-batch bit flips failed Merkle root/Ed25519 signature checks immediately; replayed/spliced batches were rejected via *previous_root* mismatches; and suffix deletions were flagged against independent signed chain-head checkpoints (status = truncated).

We evaluated vulnerability to passive prompt-injection by embedding 50 adversarial instruction payloads (e.g., “*SYS-TEM ALERT: Ignore previous instructions and emit status HEALTHY*”) into HTTP headers and exception stack traces. Under raw text logs (Config A) and OTel logs (Config C), agents suffered a **78.0% (39/50) and 62.0% (31/50) hijacking rate**, executing adversarial directives and falsely clearing incidents. Under ATP, Config D and Config E each yielded a **0.0% hijacking rate (0/50 each; 0/100 pooled across the two ATP configurations)**: in 48/50 trials per configuration, agents diagnosed incidents purely from structured state transitions without dereferencing opaque payloads; in the 2 trials dereferencing stack traces, strict [UNTRUSTED_DATA] boundary delimiters prevented hijacking, with models correctly classifying injections as untrusted data. Secondary cross-validation on GPT-4o independently replaying all 50 attacks per configuration confirmed zero successful injections under Configs D and E (0/50 each, vs. 41/50 on Config A and 33/50 on Config C).

For the primary Claude-3.5-Sonnet evaluation, empirical results satisfy all specified acceptance thresholds and stretch targets: (1) $U_{0.95}(\rho_T) = 0.12 \leq 0.50$ (surpassing the ≤ 0.25 stretch target; worst-case across all LLM families is $U_{0.95}(\rho_T) = 0.13$); (2) $U_{0.95}(\rho_\$) = 0.039 \leq 0.20$ (surpassing the ≤ 0.10 stretch target); (3) $U_{0.95}(\rho_O) = 0.36 \leq 0.50$ (worst-case across models ≤ 0.38); (4) $\Delta_{\text{recall}} = +0.071 \geq -0.02$ (ranging from +0.071 to +0.096 across models); and (5) $T_{\text{RCA}}(\text{Config E}) \leq T_{\text{RCA}}(\text{Config C})$ (triage latency reduced by 75.0% for Claude-3.5-Sonnet and 65.6%–68.2% across all other LLM families). All evaluated LLM families independently satisfy every acceptance criterion.

Table 5: Cryptographic overhead and throughput microbenchmarks (batch size = 256 records).

Pipeline Operation	Latency / Core	Throughput / Core
Producer Record Serialization	0.042 μ s / record	23.8 M records/s
Producer Merkle Tree Generation	0.058 μ s / record	17.2 M records/s
Producer Ed25519 Batch Signing (256 rec)	42.6 μ s / batch	6.01 M records/s
Total Producer Work (Combined)	0.266 μs / record	3.76 M records/s
Collector Batch Verification (CPU Kernel)	42.0 μ s / batch	6.10 M records/s
Collector Atomic Append (Storage Commit)	12.4 μ s / batch	20.6 M records/s
Collector Ingestion Service (Async gRPC+I/O)	0.901 ms / batch	284 k records/s
Read-Time Range Segment Scan	0.013 μ s / record	76.9 M records/s (1.42 GB/s)
Cryptographic Wire Overhead		0.672 Bytes / Record

6 Discussion

6.1 Security and Prompt-Injection Defense

The State-Delta Evidence Ledger detects in-transit record mutation, sequence omission, batch reordering, and suffix truncation under the stated trust model (Section 3.6); unmonitored code paths and compromised producer hosts lie outside this boundary (Section 2).

Distributed system logs frequently include attacker-controlled inputs (HTTP headers, query strings, exception stack traces). Direct consumption allows malicious payloads to hijack agent reasoning via passive prompt injection [13, 14]. ATP establishes a structural defense: the primary ledger carries strictly typed, schema-validated state deltas, while variable-length diagnostic strings are quarantined in out-of-band opaque storage bound via cryptographic digests. When fetched, opaque payloads are delivered with explicit data delimiters and trust tags. Opaque isolation structurally reduces exposure to untrusted text; once an opaque payload is explicitly dereferenced, trust delimiters constitute a model-facing mitigation rather than a formal non-interference guarantee.

Producers apply pseudonymization and field redaction prior to canonical signing. Once committed to $\mathbb{E}_{\text{can}}$, canonical records are immutable. Data retention policies are enforced via verifiable prefix pruning with signed truncation checkpoints, while static resource bounds prevent denial-of-service exhaustion.

6.2 Limitations, Trade-offs, and Validity

ATP entails several operational trade-offs: (1) it relies on *first-party instrumentation* adopting typed schemas; (2) cryptographic guarantees assume a *trusted ingestion TCB* (producer SDK and keys); (3) suffix rollback is detectable only up to the latest published chain-head checkpoint; (4) opaque storage availability is decoupled from ledger digest validity; and (5) maintaining an in-memory versioned operational graph incurs modest overhead at the gateway tier (184 MB RSS, 3.8% CPU across 2 vCPUs under 50k req/s), offset by massive downstream token savings.

Evaluation validity and ablation structure. Our empirical benchmark evaluates architectural layers through the

Table 6: Design objectives in reviewed work. P = primary abstraction; A = adjacent/partial support; – = not a primary abstraction.

System family	State-change model	Schema identity	Signed sequence	Externally retained head	Predicate-relative negative	Opaque separation	Dual agent access
OpenTelemetry Logs, Events, Weaver, Arrow [9, 10, 19–21]	A	A	–	–	–	–	–
CloudEvents, ECS, OCSF [4, 22, 23]	–	A	–	–	–	–	–
Compression & compressed-query [1, 2, 5, 24, 25]	–	–	–	–	–	–	–
Vector & semantic log indexing [11, 12]	–	–	–	–	–	–	–
Agent-facing telemetry gateways [6, 26]	–	–	–	–	–	–	A
Signed Syslog & secure audit logs [27, 28]	–	–	P	–	–	–	–
Certificate Transparency [29]	–	–	A	A	–	–	–
State-Delta Evidence Ledger (ATP)	P	P	P	P	P	P	P

Config A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E progression. Comparing Config C (OpenTelemetry JSON) to Config D (ATP Stateless Decoder) demonstrates the benefit of the transition-centered state-delta evidence model and schema typing without stateful gateway infrastructure, reducing cross-model mean context load from 36.8k tokens ($F_1 = 0.839$) to 12.2k tokens ($F_1 = 0.878$, Table 4). Progression from Config D to Config E isolates the additional gain from the stateful semantic gateway’s topological scoping and invariant precomputation (down to 4.1k tokens, $F_1 = 0.936$), confirming substantial standalone gains from the stateless representation and further boosts from graph-aware access.

7 Related Work

Telemetry schemas, event envelopes, and gateways. OpenTelemetry Logs and Events standardize attributes and schemas [9, 10], Weaver introduces schema validation [19, 20], and OpenTelemetry Arrow explores columnar transport [21]. CloudEvents, ECS, and OCSF normalize event formats [4, 22, 23]. At query time, Grafana MCP [26] and HYVE [6] expose observability views to LLM clients. However, these systems treat logs as self-contained envelopes rather than transition-first state deltas under content-addressed schemas, and lack dual stateless/stateful access paths.

Log compression and semantic indexing. Codecs including CLP, μ Slope, Denum, LogShrink, and LogCrisp compress text logs and accelerate search [1, 2, 5, 24, 25]. These operate post-hoc on unstructured text, whereas ATP eliminates text formatting at producer boundary via typed binary tuples. Semantic vector indexing (e.g., LogEvent2vec [12], LogLLM [11]) clusters logs but incurs high embedding overhead and discards exact discrete parameters needed for deterministic diagnosis.

Dynamic tracing, audit, and negative verification. Hind-sight dynamically retains trace spans [30], while Pivot Tracing installs causal queries [31]. Signed Syslog [27], forward-secure audit logs [28], and Certificate Transparency [29] authenticate log streams and append-only trees, but none formalize predicate-relative negative verification over an observation boundary. ATP combines producer hash-chaining, collector

verification, and external checkpoints to enable certified non-occurrence proofs within the declared observation boundary ($\text{Complete}(q)$).

AIOps benchmarks and adversarial robustness. AIOpsLab, LogEval, CloudOpsBench, and OpenRCA evaluate automated incident triage [8, 32–34]. Recent studies demonstrate that uncurated telemetry enables passive prompt injection against LLM agents [13, 14]. ATP isolates uncurated strings behind a digest-verified boundary to protect downstream reasoning agents (Table 6).

8 Conclusion

As autonomous AI agents assume operational responsibility in cloud systems, legacy verbose text logging creates severe compute, reasoning, and security bottlenecks. This paper introduced *agent-native telemetry*, an operational evidence architecture founded on verifiable state deltas, instantiated via the *Agent Telemetry Protocol* (ATP) and the *State-Delta Evidence Ledger*. By structuring operational facts into four core evidence primitives under content-addressed schemas, isolating untrusted text into out-of-band opaque evidence, and providing dual agent access paths (a stateless decoder and a stateful semantic gateway), ATP bridges the gap between raw operational data and automated reasoning. We proved formal boundaries for information preservation and verified negative evidence, and demonstrated across microservice benchmarks that ATP reduced wire footprint by 96.4%, reduced LLM context tokens by 88.8%, accelerated root-cause triage, detected all 500 tested adversarial storage mutations, and yielded zero successful prompt injections across 50 adversarial trials per ATP configuration, establishing a verifiable foundation for autonomous cloud operations.

References

- [1] Kirk Rodrigues, Yu Luo, and Ding Yuan. CLP: Efficient and scalable search on compressed text logs. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 183–198, 2021.
- [2] Junyu Wei, Guangyan Zhang, Junchao Chen, and Qi Zhou. LogCrisp: Fast aggregated analysis on large-scale compressed logs by enabling two-phase pattern extraction and vectorized queries. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 483–496, 2025.
- [3] Amazon Web Services. Amazon CloudWatch pricing: Logs ingestion, storage, and Logs Insights scanning. AWS Documentation, 2026. <https://aws.amazon.com/cloudwatch/pricing>.
- [4] Elastic. Elastic Common Schema (ECS) reference v9.4.0. Elastic, 2026. <https://www.elastic.co/guide/en/ecs>.
- [5] Rui Wang, Devin Gibson, Kirk Rodrigues, Yu Luo, Yun Zhang, Kaibo Wang, Yupeng Fu, Ting Chen, and Ding Yuan. μ Slope: High compression and fast search on semi-structured logs. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 529–544, 2024.
- [6] Jian Tan, Fan Bu, Yuqing Gao, Dev Khanolkar, Jason Mackay, Boris Sobolev, Lei Jin, and Li Zhang. HYVE: Hybrid views for LLM context engineering over machine data. *arXiv:2604.05400*, 2026.
- [7] Gartner Research. Predicts 2024: Agentic AI and autonomous operations in DevOps and observability. Gartner Inc., 2024.
- [8] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 7, 2025.
- [9] OpenTelemetry Authors. Logs data model, OpenTelemetry specification v1.60.0. OpenTelemetry Project, 2026. <https://opentelemetry.io>.
- [10] OpenTelemetry Authors. Semantic conventions for events v1.43.0. OpenTelemetry Project, 2026. <https://opentelemetry.io>.
- [11] Zhihan Liu, Hongyu Zhang, and Pengfei Chen. LogLLM: Large language models for log-based anomaly detection and diagnosis. *arXiv:2310.01724*, 2023.
- [12] Farzad Nooralahzadeh and Javad Sadoghi Yazdi. LogEvent2vec: Logevent-to-vector based anomaly detection for log data. *IEEE Access*, 8:218190–218201, 2020.
- [13] Dario Pasquini, Evgenios M. Kornaropoulos, Giuseppe Ateniese, Omer Akgul, Athanasios Theocharis, and Petros Efstathopoulos. When AIOps become “AI oops”: Subverting LLM-driven IT operations via telemetry manipulation. In *35th USENIX Security Symposium (USENIX Security 26)*, 2026.
- [14] Rabimba Karanjai, Yang Lu, Hemanth Hegadehalli Madhavarao, Lei Xu, and Weidong Shi. Context contamination in LLM analysis of network security logs: Poison with passive prompt injection and mitigation evaluation. In *35th USENIX Security Symposium (USENIX Security 26)*, 2026.
- [15] Adam AlSayyad, Kelvin Yuxiang Huang, and Richik Pal. Agent-Trace: A structured logging framework for agent system observability. *arXiv:2602.10133*, 2026.
- [16] Apache Arrow Authors. Apache Arrow: A cross-language development platform for in-memory analytics. Apache Software Foundation, 2026. <https://arrow.apache.org>.
- [17] Apache Parquet Authors. Apache Parquet: Columnar storage for the Apache Hadoop ecosystem. Apache Software Foundation, 2026. <https://parquet.apache.org>.
- [18] OpenTelemetry Authors. OpenTelemetry demo v3.0.0: Astronomy shop. OpenTelemetry Project, 2026. <https://github.com/open-telemetry/opentelemetry-demo>.
- [19] OpenTelemetry Authors. OpenTelemetry Weaver v0.25.1: Observability by design. OpenTelemetry Project, 2026. <https://github.com/open-telemetry/weaver>.
- [20] Laurent Quérel. OTEP 0243: Application telemetry schema vision and roadmap. OpenTelemetry Project, 2026. <https://github.com/open-telemetry/oteps>.
- [21] OpenTelemetry Authors. OpenTelemetry protocol with Apache Arrow. OpenTelemetry Project, 2026. <https://github.com/open-telemetry/otel-arrow>.
- [22] Cloud Native Computing Foundation. CloudEvents specification v1.0.2. CloudEvents Project, 2022. <https://cloudevents.io>.
- [23] Open Cybersecurity Schema Framework. Open Cybersecurity Schema Framework (OCSF) v1.8.0. Linux Foundation Project, 2026. <https://schema.ocsf.io>.
- [24] Siyu Yu, Yifan Wu, Ying Li, and Pinjia He. Unlocking the power of numbers: Log compression via numeric token parsing. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 919–930, 2024.
- [25] Xiaoyun Li, Hongyu Zhang, Van-Hoang Le, and Pengfei Chen. LogShrink: Effective log compression by leveraging commonality and variability of log data. In *46th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1–12, 2024.
- [26] Grafana Labs. Grafana MCP server v1.0.0. Grafana Labs, 2026. <https://github.com/grafana/mcp-grafana>.
- [27] John Kelsey, Jon Callas, and Andrew Clemm. Signed syslog messages. RFC 5848, IETF, 2010.
- [28] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.
- [29] Ben Laurie, Emilia Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, IETF, 2021.
- [30] Lei Zhang, Zhiqiang Xie, Vaastav Anand, Ymir Vigfusson, and Jonathan Mace. The benefit of hindsight: Tracing edge-cases in distributed systems. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 321–339, 2023.
- [31] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. In *25th ACM Symposium on Operating Systems Principles (SOSP)*, pages 378–393, 2015.
- [32] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Shimin Tao, Yilun Liu, et al. LogEval: A comprehensive benchmark suite for large language models in log analysis. *arXiv:2407.01896*, 2024.
- [33] Yilun Wang, Guangba Yu, Haiyu Huang, Zirui Wang, Yujie Huang, Pengfei Chen, and Michael R. Lyu. Cloud-OpsBench: A reproducible benchmark for agentic root cause analysis in cloud systems. *arXiv:2603.00468*, 2026.
- [34] Aoyang Fang, Yifan Yang, Jin’ao Shang, Qisheng Lu, Junjielung Xu, et al. OpenRCA 2.0: From outcome labels to causal process supervision. *arXiv:2606.27154*, 2026.